Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Point of Contact:
Edward C. Zimmermann **E-mail**: <ezimmermann@acm.org>

Abstract / Summary

*CoreQuarry is a hybrid knowledge retrieval engine which emerged in 2026 from Project Schmate (שמאטע) for **re-Isearch**. It unifies lexical, structural, and semantic search into a single, high-performance platform. Unlike existing vector databases or traditional search engines, it supports true positional indexing, structure-aware queries, and typed object retrieval, enabling precise and contextually-aware search over heterogeneous document corpora. By leveraging memory-mapped, append-only indexes and a two-tier address-based caching system, the engine achieves extremely low memory footprints while scaling to handle complex, hybrid RAG queries on consumer hardware, including laptops and edge devices.*

CoreQuarry Innovation:

- Rich Operator Support: Full set of binary and unary operators (AND, OR, XOR, NEAR, BEFORE/AFTER, PEER, distance-based, field-specific operations) allow precise relational queries beyond traditional BM25-based ranking.
- Hybrid & Extensible Indexing: Lexical, structural, numeric, geospatial, phonetic, and vector types are first-class citizens; plug-ins allow user-defined types and new algorithms without redesigning the core.
- Designed to run smoothly on a single laptop or small edge
- Efficient Incremental Indexing: Append-only design with versioned updates, delete flags, and background garbage collection supports concurrent search and continuous ingestion, making it suitable for live AI pipelines.

General Impact:

This engine enables privacy-preserving, local-first AI retrieval, supporting applications in legal research, medical knowledge, industrial edge AI, and large-scale document analytics. Its small footprint and high flexibility make it uniquely suited for embedded devices, laptops, and offline environments, dramatically lowering the barrier for organizations to implement robust AI-powered search without relying on cloud services.

Key Product Differentiators

- Unified Hybrid Retrieval: Combines lexical, structural, and semantic (vector) search in a single engine.
- Rich Query Operators: Supports advanced positional, proximity, field-specific, and relational operators beyond traditional BM25.
- Typed Object Indexes: Indexes dates, numbers, hashes, geospatial, phonetic, and vectors as first-class data types.
- Append-Only, Memory-Mapped Architecture: Enables concurrent search and continuous ingestion with minimal RAM usage (lexical-only: min ~8MB).
- Extensible & Pluggable: Supports new document types, custom fields, and alternative retrieval algorithms (HNSW, NSG, etc.).
- Local-First Deployment: Runs efficiently on laptops and edge devices, enabling privacy-preserving AI search without cloud dependencies.
- High Throughput: Demonstrated thousands of QPS on consumer hardware for complex hybrid queries.
- Extremely efficient and fast HNSW implementation with support for quantization and pass-through.

The system solves three major problems that current RAG and search solutions don't:

1. Hybrid Retrieval at Scale
    - Lexical + structural + semantic in a single index

    - True boolean/proximity operators and field-scoped searches
    - Optional vector embeddings for AI-enhanced relevance
2. Local-first & Lightweight
    - Runs fully on consumer hardware (including laptops, edge devices)
    - Lexical-only mode: <10MB RAM
    - Support for CUDA, Metal, Vukan etc. for embeddings. Best in class performance.
    - No dependency on heavy external services (no Elasticsearch, no FAISS clusters)
3. Extensibility & Heterogeneous Data
    - Typed object indexes (dates, numbers, geospatial, phonetics, hashes, vectors)
    - Plug-ins for custom document formats and schemas
    - Supports structured traversal, wildcards, field/path scoping

Potential Markets / Applications

1. Local AI Assistants
    - Law firms, medical practitioners, researchers: local RAG over sensitive documents
2. Edge AI Devices
    - IoT devices, robotics, industrial sensors: offline retrieval of structured datasets
3. Enterprise Knowledge Infrastructure
    - Replace multi-system RAG stacks (vector DB + search engine + pipeline)
    - Offers privacy, low-latency retrieval, and structured queries
4. Archival & Compliance
    - Historical corpora, standards, legal/financial archives: structural search + temporal/versioning
5. Developer Tooling
    - Embed directly into applications like SQLite: “SQLite for AI retrieval”

On Premise Self-Hosting on commodity accelerator hardware versus Cloud

Many institutions manage strict requirements for data stewardship, legal compliance, and long-term digital preservation. While cloud solutions are convenient, running knowledge retrieval and RAG architectures wholly on-premise protects institutional integrity. Because these local pipelines are specifically designed to run on affordable, consumer-off-the-shelf (COTS) systems, organizations can achieve total data independence without a massive capital investment.

Crucially, an on-premise architecture ensures seamless compliance with a web of rigorous regulatory frameworks:

- **The General Data Protection Regulation (GDPR):** Local hosting inherently satisfies core **GDPR** mandates—such as Data Minimization (Article 5) and strict restrictions on International Data Transfers (Chapter V)—by eliminating the risk of personal data leaving sovereign borders.
- **National and Local Protection Acts:** It preserves strict adherence to localized adaptations, such as the German **Federal Data Protection Act (BDSG)**, and national public record acts like the UK **Public Records Act 1958** or the US **Federal Records Act (FRA)**.
- **Specialized Archival and Ethical Standards**: Processing records locally aligns with the International Council on Archives (ICA) Code of Ethics, which mandates that archivists actively protect corporate and personal privacy within electronic records.

By optimizing these systems for high-performance, consumer-off-the-shelf (COTS) hardware, institutions can deploy advanced local AI without requiring prohibitive capital expenditures. This approach ensures total data sovereignty and long-term digital preservation while maintaining strict adherence to public procurement budgets.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

The "Hybrid-Local" Trap: Hidden Cloud Dependencies
However, when evaluating self-hosted knowledge retrieval systems, organizations must look beyond superficial marketing terminology to verify true data isolation. Many software solutions marketed as "on-premise" operate on a hybrid architecture that quietly relies on external cloud APIs—such as OpenAI or Cohere—for critical processing stages like text embedding and vectorization. While the core application interface may reside on a local server, the system continuously transmits raw institutional data to offshore hyperscalers to generate these mathematical representations.

For organizations bound by strict privacy mandates, this architectural dependency represents a critical vulnerability. Passing data to a third-party API invalidates the compliance, confidentiality, and security benefits of a local deployment. To maintain absolute data integrity, the entire pipeline—including document ingestion, text chunking, embedding generation, vector storage, and large language model (LLM) inference—must run completely contained within the local COTS environment.

Absolute Digital Preservation and Long-Term Stability

- **No Vendor Lock-In:** Reliance on a cloud provider risks catastrophic disruption if the vendor raises prices, alters their API, or goes out of business.
- **Immutable Infrastructure:** On-prem code bases can be frozen in time, ensuring that search and retrieval tools remain functional and identical for decades.
- **Offline Resiliency:** Libraries must function during regional internet outages or infrastructure failures; local setups ensure internal search operations never go offline.

Financial Predictability and Zero Usage Fees (FinOps)

Publicly funded institutions operate on fixed, rigid annual budgets that do not align with unpredictable cloud billing models.

- **No Per-Query API Fees:** Cloud RAG systems charge per token or per search volume, leading to volatile costs if a search tool suddenly goes viral with patrons.
- **Fixed Capital Expenditure:** Purchasing on-prem hardware is a predictable, one-time expense that fits traditional institutional grant structures and capital budgets.
- **Leveraging Existing Hardware:** Libraries can repurpose existing local server infrastructure or institutional university clusters to run highly optimized, quantized open-source models.

Uncompromising Customization and Domain Accuracy

Standard cloud search tools and commercial AI models are optimized for general web data, not specialized historical archives.

- **Niche Metadata Integration:** On-prem systems allow direct injection of highly specific library standards (like MARC21, Dublin Core, or custom taxonomies) directly into the retrieval pipeline.
- **Historical Language Tuning:** Local open-source models can be custom-tuned to read archaic spellings, dead languages, or specific regional dialects without cloud filtering mechanisms blocking the content.
- **Algorithmic Transparency:** Researchers require objective, un-biased search results; local open-source retrieval pipelines allow staff to inspect the exact retrieval math, proving how an answer was generated.

Power and Energy Efficiency
Designed and optimized for COTS unlike enterprise data center hardware which demand three-phase industrial power grids and specialized server room water chillers.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Support:

Through the NGI0 Discovery Fund, a fund established by NLnet with financial support from the European Commission's Next Generation Internet programme, under the aegis of DG Communications Networks, Content and Technology under grant agreement No 825322.

Through the NGI0 Commons Fund, a fund established by NLnet with financial support from the European Commission's Next Generation Internet programme, under the aegis of DG Communications Networks, Content and Technology under grant agreement No 101135429. Additional funding is made available by the Swiss State Secretariat for Education, Research and Innovation (SERI).

Through a Grant from the Bundesministerium für Forschung, Technologie und Raumfahrt (Germany) GRANT_NUMBER: 01IS22S32 (exploring support of the IPFS and supporting remote indexing). Through a grant from the European Commission Coordination and Support Action (CSA) on ICT standardisation (extending support for additional post ISO-8601:2019 features).

Additional support was provided by a grant from OpenData CH/Mercator Foundation CH.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Appendix I: Technology Backgrounder / The 30 year old new kid

re-Isearch is the modern evolution of iSearch—the pioneering open-source engine of the mid-90s we co-developed with CNIDR. Originally developed to solve the "document type" problem of the early web, the technology was reborn through the support of NLNet as well as grants from BMFTR (German Federal Ministry for Research, Technology and Space), Open Data Switzerland and the Mercator Foundation. Today, we've moved beyond lexical indexing adding with Project Schmate vector indexes to become CoreQuarry. It combines thirty years of architectural expertise with cutting-edge hybrid search to help mine the core of human knowledge.

Our philosophical ancestor is early SGML retrieval systems (like OpenText which grew out of a University of Waterloo project from Timothy Bray, Frank Tompa, and Gaston Gonnet to digitize the Oxford English Dictionary) but designed to support beyond SGML multiple document formats and data types; and modernized for:

- embeddings
- local-first compute
- RAG

For vector search we use the The Hierarchical Navigable Small World (HNSW) algorithm. Introduced in 2016 by Yu. A. Malkov and D. A. Yashunin in their paper, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs". It is the evolutionary successor to the Navigable Small World (NSW) graph, combining it with the hierarchical structure of Skip Lists.

Both our algorithms for lexical and vector structures were designed to move beyond simple linear scans. HNSW achieves logarithmic complexity $O(\log N)$ for vectors, while PatArrays provide similar efficiency for finding any substring in a massive corpus without an inverted index.

Hardware has progressed dramatically since the mid-1990s when those SGML retrieval systems were deployed. Back then:
- high end servers (e.g. Sun Ultra Enterprise, IBM RS/6000 SP, HP 9000, DEC Alpha servers, and high-end Intel Pentium Pro servers): had 1-4 GB RAM (mainly EDO or perhaps early ECC DRAM). The memory bus speed was ~60-66 Mhz and the bandwidth was ~200-500 MB/s. Latency was ~60-80ns.
- High-end disk arrays: 50–500 GB total (e.g., an EMC Symmetrix could hold ~400 GB) where Individual SCSI drives: 2–18 GB per spindle and RAID controllers with cache: 32–256 MB write cache

The Numbers Side-by-Side

Metric	Mid-1990s Enterprise	Modern 2026 Workstation	Improvement
RAM capacity	1–4 GB	128 GB–2 TB	~100–500×
Memory bandwidth	~400 MB/s	~300 GB/s	**~750×**
Memory latency	~70 ns	~75 ns end-to-end	Roughly flat
Disk seq. read	10–15 MB/s	12,000–14,000 MB/s	**~1,000×**
Disk random IOPS	100–200	1,000,000–2,000,000	**~10,000×**
Disk seek/latency	8–12 ms	0.02–0.1 ms	**~100–500×**
Storage capacity	400 GB (array)	4–8 TB (single drive)	~10–20× per drive
CPU clock	100–500 MHz	4,000–6,000 MHz	~10–60×

A fully configured mid-1990s enterprise server with 2–4 GB RAM, a modest disk array (~100 GB), RAID, and software licenses: $500,000 – $3,000,000+ (in 2026 dollars that's roughly $1M – $6M) **versus** ~$8,000–$9,000 for a 2026 high-end workstation and even under $2000 for a fast consumer desktop (64-core, 64GB DDR5-6000 RAM, 2TB SSD etc). Even the €475 (Netto) Apple M4 Mini (standard M4 chip) offers 38 TOPS, 120GB/s memory bandwidth, 16GB RAM and 255GB SDD (under 0.1 ms seek latency). The

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

NVIDIA DGX Spark, powered by the GB10 Grace Blackwell SOC, delivers up to 1 PetaFLOP, memory bandwidth of approximately 273 GB/s, 128 GB of unified LPDDR5x memory and 4GB SSD for 4.349,00 €

Example of an Enterprise Ready Specification (Prosumer Class):

Component	Target Specification	Institutional Rationale	Estimated Cost (USD)
Graphics Processing Units (GPUs)	**2x NVIDIA GeForce RTX 5090** (32GB GDDR7 VRAM per card)	Provides a massive **64GB of combined VRAM** pool over a high-bandwidth PCIe interface. This capacity natively holds a 70B parameter model at Q4_K_M quantization with enough residual VRAM to support expansive context windows and user sessions.	$4,000 – $4,600
Processor (CPU)	**AMD Ryzen Threadripper 7960X** (24 Cores / 48 Threads) or high-end **Intel Xeon W-2400**	Provides the raw parallel processing power required for the document ingestion, optical character recognition (OCR), text tokenization, and vector indexing phases of a RAG pipeline.	$1,300 – $1,500
System Memory (RAM)	**128GB to 256GB DDR5 ECC** (4x 32GB or 4x 64GB Multi-Channel Kits)	Houses the runtime footprint of the vector database (e.g., Qdrant, Milvus) and provides a secure memory buffer for processing massive archive batch imports. Error-Correcting Code (ECC) RAM prevents memory leaks and bit-flips during multi-day background indexing tasks.	$600 – $1,000
Storage (SSD)	**4TB NVMe M.2 SSD**(PCIe Gen 5, e.g., Crucial T700 or Samsung PRO series)	Yields high-speed read/write performance (>12,000 MB/s). Essential for instantaneous **zero-copy memory mapping (mmap)** of multiple GGUF files and quick structural queries into the local vector index.	$350 – $500
Motherboard & Power	Workstation Motherboard (WRX90 / PCIe Gen 5 x16/x16 split) + **1600W 80+ Titanium PSU**	Requires native physical spacing and dual x16 lane routing to prevent GPU bottlenecking. The industrial-grade power supply unit guarantees continuous power stability under heavy, concurrent processing loads.	$800 – $1,100
Cooling & Chassis	High-Airflow E-ATX Workstation Case + Premium Liquid/Air Cooling Systems	AI inference generates intense thermal output over prolonged periods. High-static pressure fans prevent hardware throttling, ensuring deterministic system speeds during heavy usage sessions.	$300 – $500
Total Estimated Capital Expense		**Enterprise-grade on-premise infrastructure safely under budget limits.**	**$7,350 – $9,200**

This specification allows the institution to host a large-scale, quantized 70B parameter model (such as Llama 3 70B or Qwen 3 70B) natively alongside an embedding model.

Form Factor: Housed in a standard standalone tower or a compact 4U rackmount chassis, avoiding the need for dedicated data-center space rental.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

What Contemporary Hardware Means for Information Retrieval

The most dramatic difference isn't raw speed — it's random I/O latency and IOPS, which is exactly what database queries and information retrieval hammers hardest.

- A full-table scan that took 10 minutes on a 90s SCSI array takes seconds on NVMe
- A database query requiring 10,000 random disk reads: 90s machine takes 80–120 seconds just in I/O wait; modern NVMe does it in ~10–50 milliseconds
- The entire working dataset that required careful buffer pool management in the 90s (because RAM was scarce) now fits entirely in RAM on a modern system — eliminating disk I/O almost entirely for hot data
- Memory bandwidth improvements mean in-memory analytics (columnar stores, etc.) can process data ~750× faster even if latency is similar

The 90s DBAs spent enormous effort on I/O optimization— index tuning, careful buffer management, disk striping — precisely because every random read cost 10ms. On modern hardware, that same problem is largely dissolved. The bottleneck has moved entirely to CPU cache and memory access patterns.

What this means is that algorithms like the inverted index are obsolete! Their historical advantage (accepting their significant limitations) makes no sense in 2026.

The inverted index was an engineering response to hardware constraints, not an ideal data structure:

- Disk was slow and expensive— you couldn't afford to scan raw text
- RAM was tiny— you couldn't hold documents in memory
- IOPS were precious— every random read cost 10–12ms
- So you precomputed posting lists to minimize disk seeks at query time

The entire architecture mantra of Lucene, early Google, and most 1990s IR systems was essentially: "*disk is our enemy, precompute everything possible to avoid touching it.*"

Patricia Trees and Pat Arrays Fell Out of Favor in the 1990s because of hardware:

- Pointer chasing is catastrophically cache-unfriendly— every node traversal is a potential cache miss
- On 1990s hardware, a cache miss meant going to main memory at 60–80ns, and with tiny caches (L1 was 8–16KB) misses were constant
- The inverted index, despite its bulk, had more predictable, sequential access patterns that could be optimized with careful disk layout
- Patricia trees also consumed significant RAM for pointers — precious in an era of $50,000/GB RAM
- Worst case pointer density: a tree over a million terms might have millions of pointer pairs, each potentially a cache miss during traversal

So Patricia trees were theoretically elegant but practically punished by memory hierarchies of the era.

Cache hierarchies have transformed things

Cache	1995 Size	2026 Size	Improvement
L1	8–16 KB	32–64 KB	~4×
L2	256 KB (rare)	512 KB–1 MB per core	~4–8×
L3	Nonexistent/tiny	64–256 MB (Threadripper)	∞ → huge
RAM	256MB–4GB	128GB–2TB	~500×

A Patricia tree over a large IR vocabulary (~1–10 million terms) occupies perhaps 500MB–2GB of RAM.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

On a modern system this fits entirely in L3 cache on high-end CPUs, or at worst in RAM with DDR5's enormous bandwidth.

When the whole tree lives in L3, pointer chasing is no longer going to slow DRAM — it's hitting ~10 cycle L3 latency vs the ~200 cycle DRAM penalty that killed Patricia trees in the 1990s.

PAT Array Problem (1992)	**2026 Solution**
Index too large for RAM	128GB–2TB RAM — fits comfortably
O(log n) random accesses slow	DDR5 random access ~75ns, L3 ~10ns — negligible
Construction too slow	Modern O(n) suffix array construction (SA-IS algorithm) + fast CPUs
Binary search cache-unfriendly	Entire index in L3/RAM — cache misses collapse
No word-level ranking (BM25 etc.)	Can layer neural/embedding ranking on top

Our indexing architecture builds on ideas related to Patricia trees and Pat arrays, but extends them to address the I/O and scalability constraints that have historically limited their use in large-scale information retrieval. A Patricia tree provides an efficient compressed trie structure for prefix searches, but it requires pointer-heavy navigation and does not map well to modern memory-mapped storage or high-throughput disk access. Pat arrays, by contrast, store lexicographically ordered suffix or term references in contiguous arrays, improving locality but traditionally requiring the underlying corpus text to be stored alongside the index and incurring expensive disk reads during lookup. Our approach separates corpus storage from the index while maintaining compact addresses.

Caching Strategy based on Word Length Distribution

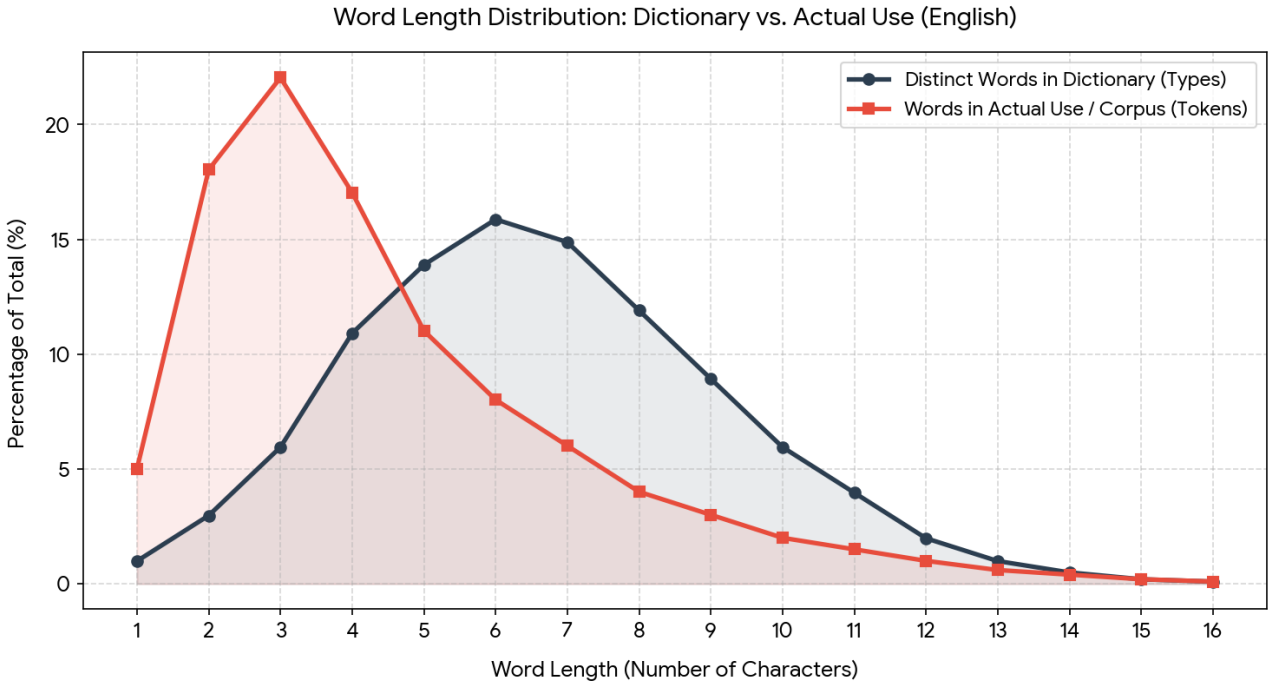
Information Efficiency: Human communication is optimized for speed. According to **Zipf's Law of Abbreviation**, the words we repeat most frequently are compressed by evolution into the shortest possible forms

Distinct Words (Types) follow a **unimodal distribution** (a hump-shaped curve). The vast majority of dictionary words are moderately long (typically 6–9 characters in English) because a language needs permutations of letters to create a unique vocabulary. Very short words are rare because there are only so many combinations you can make with 1 to 3 letters

Words in Use (Tokens) follow a **monotonic decrease or highly right-skewed distribution**. When analyzing a real book or corpus, massive frequency is dominated by short grammatical words ("the", "of", "and", "to"). This causes a huge spike at lengths 2–4, dropping sharply as words get longer.

Grammar vs. Content: Short words (2–4 letters) act as structural or grammatical "bridges" (e.g., pronouns, prepositions, articles). We reuse them constantly in every sentence, whereas long words (8+ letters) carry specific technical or descriptive content and are only used when strictly necessary

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.



Our 2-tier cache, originally introduced in our proprietary fork, allows the advantages of the separate corpus but removes the need to read it for words less than some pre-set prefix length.

This works remarkably well especially in English

- ➔ 1–7 letters: ~82.97%
- ➔ 8–10 letters: ~13.46%
- ➔ 11–14 letters: ~3.46%
- ➔ 15 letters: ~0.076%
- ➔ ≥16 letters: ~0.034% combined

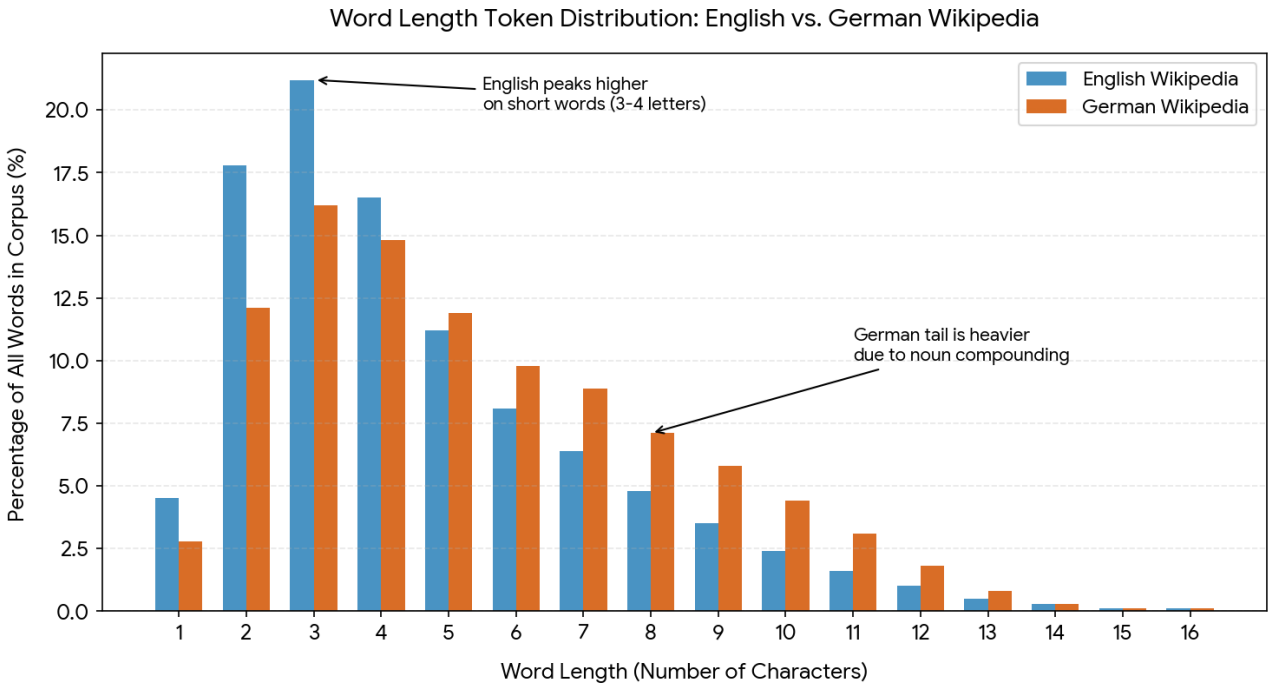
≈ 99.9% of words are under 15 characters long in typical English text. **10 characters or less: ~96.4%.** A 10-character prefix is long enough that collisions are uncommon, so the resulting count will be close to the original dictionary size.

So even with a length set to 10, 96% or all words won't demand a corpus lookup. This cache is also relatively small since it would require only 26 bytes per unique word (for 64-bit indexes).

Source	Count
Words ≤10 chars	~118k
Unique prefixes from longer words	~44k
Total unique entries	**~162k**

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

≈150,000 – 170,000 max unique entries so we can safely assume that the maximum size of the cache will be under 4.5 MB.



German behaves very differently from English because of productive compounding (e.g., *Krankenhausverwaltungsgesetz*). So truncating to 10 characters collapses many more words.

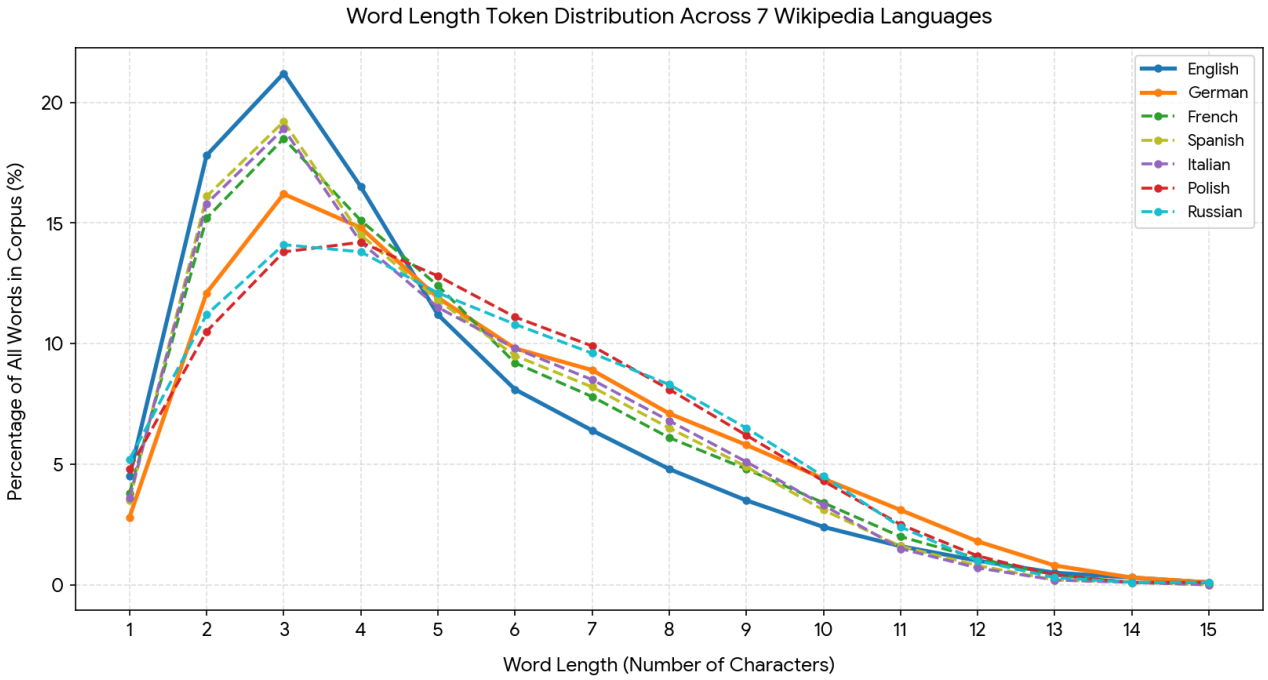
Length	Approx. share
≤10 letters	~40–50%
11–15 letters	~30–35%
16+ letters	~20–30%

Expanding this analysis across **three different language families** (Germanic, Romance, and Slavic) reveals clear patterns in how grammatical structures dictate token word length.

Comparative Insights by Language Family

- **The Romance Family (French, Spanish, Italian)**: These three closely mirror each other. They behave like English with a strong peak around **3 to 4 characters**, heavily driven by shared grammatical features like short articles (*el*, *la*, *le*, *les*, *il*, *i*), prepositions (*de*, *di*, *en*), and conjunctions (*y*, *et*, *e*).
- **The Slavic Family (Polish, Russian)**: Slavic languages completely lack definite and indefinite articles (no words for "a" or "the"). Because they skip these highly repetitive ultra-short words, their curves are much flatter and **shifted to the right**, peaking broader around **3 to 6 characters**. They also rely on extensive case-inflected suffixes, lengthening their baseline vocabulary.
- **The Germanic Family (German vs. English)**: While English behaves like a Romance language in its word length due to heavy reliance on short particles, German stands out completely alone with its prolonged, thick tail spanning from **7 to 12+ characters** due to its compound word architecture.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.



Practical rule for optional prefix length according to language for 95% unique

Language	Typical lexicon size	Safe prefix length	Main reason
English	~170k	**8–10**	Short words, low inflection
German	~350k	**12–14**	Strong compounding
French	~200k	**9–11**	Shared Latin prefixes
Spanish	~200k	**9–11**	Verb inflection
Italian	~180k	**9–11**	Romance morphology
Russian	~200k	**10–12**	Prefix + case system
Swedish	~250k	**11–13**	Compounding
Greek	~200k	**11–13**	Rich inflection, long stems
Polish	~250k	**12–14**	Very rich case morphology

If one needs a prefix length that works well across all these languages:

Prefix length	Result
10 chars	Works well for English / Romance languages
12 chars	Good for most European languages
14 chars	Safe even for German, Polish, Swedish

✅ Best universal compromise: 12 characters

This usually keeps >90–95% of entries unique across European languages. In a mixed environment we can expect the cache to be under 8MB.

Memory Layout Optimization

- Phrase queries (term A followed by term B within distance d) become a merge of two contiguous array slices — elegant and cache-friendly
- Co-occurrence statistics can be computed with a single sequential pass
- Compression (delta encoding on doc_ids, positions) works extremely well on sorted runs — better than inverted index compression in many cases

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

•The whole structure is memory-mappable— OS handles paging, no custom buffer pool needed

Field/Path complexity for Lexical Search. Looking at TEI P5 guidelines as an extreme example with a very large vocabulary.

- ~600+ distinct elements (tags) in the full schema
- **~250–300 attributes**
- Elements are grouped into modules (core, header, verse, drama, manuscript description, dictionaries, etc.)

However, no real TEI file uses all of them. No more than 100–200 distinct tags is common in a complex scholarly TEI document.

Typical ranges seen in TEI corpora:

Project complexity	Unique paths
Basic TEI text	100–300
Scholarly edition	300–800
Rich digital humanities corpus	800–1500
Large TEI aggregation	1500–3000+

So a single complex TEI document might easily have ~300–800 unique paths.

There are, of course, more complex document models than TEI.

1) **JATS (Journal Article Tag Suite)**

- Used by PubMed Central.
- Very deep citation, funding, and metadata structures.
- Often 200+ tags per article.

2) HL7 CLINICAL DOCUMENT ARCHITECTURE

One of the most complex real-world XML standards.

- massive schema
- heavy metadata
- controlled vocabularies
- deep nesting

Often thousands of unique paths across medical documents.

3) LEGAL XML / AKOMA NTOSO

Extremely complex legislative documents:

- hierarchical law structures
- cross references
- amendments
- semantic tagging

With current hardware in 2026, fast binary search, in absolute worst case, over 800 memory mapped files is still relatively performant: $O(N \log(M))$ where N is the number of indexes (here 800) and M is the maximum number of items in each. But since the indexes are sorted it is more typically: $O(\log(NM))$

Most path searches will be into just a few fields (tags) or paths whence a fast binary search in just a few memory mapped and cached files. As noted above: DDR5 random access ~75ns, L3 ~10ns — negligible.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Key Considerations:

- Mapping requires system calls (open + mmap) to map a file into the process's virtual address space. If the file's metadata is already in the OS page cache, the open() call is exceptionally fast.
- High-Speed Storage (NVMe/Storage Class Memory): The cost of the page fault is increasingly determined by the latency of the underlying SSD, which is extremely fast on modern systems.
- 64-Bit Addressing: There is virtually no limit on mapping file sizes on 64-bit systems.
    - Cached Performance: If the file is already in the page cache, the mmap() overhead is minimal
- VMA Limits: While Linux has a default limit on the number of memory map areas a single process can have (vm.max_map_count, often 65,530) this is well below max fields/paths.
- The Unix kernel's page cache manages memory-mapped files lazily.
- Metadata Efficiency: The OS is very efficient at opening a few files as needed.
- By mapping only the files (lazily on demand) as they are actually needed we improve, given how the kernel uses variations of LRU to manage the page cache, efficiency.

The advantage of the this approach is that memory demands don't spiral out of control even with an "excessive" number of fields. Since CoreQuarry only maps (using a LRU heuristic) the fields being searched memory demands for searches is modest. During index the number of fields/paths has no impact on consumption since indexing buffers are pre-defined irrespective of their numbers—relevant only to the maximum size of an individual document.

Appendix II: What is different?

70% of Retrieval-Augmented Generation (RAG) systems fail in production[1]. They mostly fail at retrieval, not generation. The main issues are 1) context mismatch (chunking) 2) semantic collapse[2] 3) drift. Handling semantic drift is a "missing link" for Retrieval-Augmented Generation (RAG) systems because it represents the silent degradation of retrieval accuracy over time, turning previously reliable AI systems into sources of outdated or conflicting information. As enterprise data evolves, the "meaning-space" of the vector database shifts, causing embeddings of new documents to diverge from old ones, or queries to no longer align with stored knowledge. When a RAG system is updated with new documents without re-embedding old data or retraining the retriever, it creates a "single vector index" with mixed, conflicting, and divergent semantic contexts, known as **Knowledge Drift**.

Unlike a traditional code error that breaks a system, drift causes the RAG system to continue operating, but with silently declining accuracy. This degrades trust as the system provides subtly outdated or irrelevant information.

To address these issues, DeepQuarry (R&D), implements:

- **Continuous Monitoring:** Actively monitoring for "Retrieval Misses" and using specialized metrics to detect drift.
- **Semantic Segmentation:** segmenting data into more coherent semantic clusters working backwards from detected drift
- **Incremental Updating:** Re-indexing or re-embedding data with updated models.
- **Hybrid Search:** Combining semantic vector search with keyword search to catch specific, updated terminology that embeddings might miss.
- **Semantic Chunking:** Breaking down documents based on logical structure (not arbitrary character counts) to maintain the semantic integrity of each segment. The basic concept is that the explicit and implicit structure of documents anchors context.
- **Semantic indexes by field and sharding**: this avoids the network getting too crowded. When everything is thrown into a single pile the evidence is that starting as early as with 10k documents

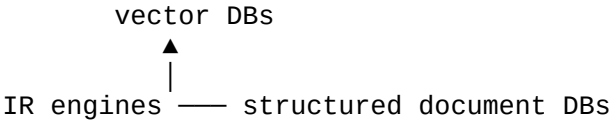
1 https://python.plainenglish.io/why-70-of-rag-implementations-fail-and-the-6-things-that-separate-production-grade-systems-97501c11b682

2 https://dho.stanford.edu/wp-content/uploads/Legal_RAG_Hallucinations.pdf

things start to fall part. The architecture implemented in CoreQuarry vastly extends the document capacity.

Appendix III: Competitive Landscape

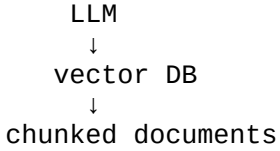
Our architecture uniquely overlaps three domains simultaneously:



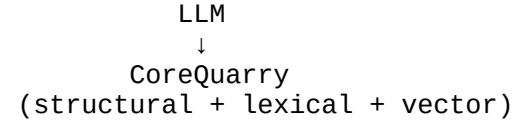
The engine has several advantages commercially:

- runs locally
- small footprint
- high performance
- hybrid retrieval
- structured document awareness

Right now, RAG infrastructure is mostly:



This architecture is deeply flawed. Instead or model:



If positioned correctly, it could disrupt the market become the retrieval layer for local AI systems.

1) PRIMARY DIRECT COMPETITORS (COMBINING LEXICAL + VECTOR SEARCH)

Elasticsearch

- Lucene-based
- supports BM25 + vector search
- widely used in RAG stacks

Comparative Weakness:

- poor structural awareness
- heavy infrastructure
- BM25-centric ranking
- Written in Java
- Triple license / AGPL

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

OpenSearch

Fork of Elasticsearch.

Weakness:

- same Lucene constraints

Weaviate

- hybrid search
- object store
- semantic queries
- BSD 3-Clause license.

Weakness:

- vectors first
- weak lexical IR
- heavy infrastructure
- written in Go. This means some memory is not immediately available for reuse when it is no longer needed.

Qdrant

- fast ANN (HNSW)
- filtering
- Apache 2.0

Weakness:

- minimal lexical search
- no structural IR

Milvus

Heavily-modified forks of third-party open source similarity search libraries, such as FAISS, DiskANN and HNSWlib.

- large-scale vector DB (Kubernetes clusters)
- Apache 2.0

Weakness

- not really a document retrieval engine

The CoreQuarry Competitive Edge over Unified Hybrid & Vector Engines: Direct competitors in the modern RAG and semantic search space fall into two deeply flawed architectural camps: **Lucene-derived heavyweights** (Elasticsearch, OpenSearch) trying to retroactively bolt vector tracking onto legacy inverted frameworks, and **vector-first specialized databases** (Weaviate, Qdrant, Milvus) struggling to provide robust, deep lexical text capabilities.

Across both categories, these systems demand massive infrastructure footprints, rely on heavy, cache-indifferent language runtimes (Java/Go) that suffer from garbage collection latency, and map data using high-overhead object-layer abstractions.
CoreQuarry completely breaks this paradigm through a **hardware-sympathetic, unified C++ design**:

•**Zero Inverted-Index Bloat:** Instead of the immense RAM and storage overhead required by Lucene's term-dictionaries, CoreQuarry utilizes a proprietary, highly compact implementation of **Patricia Arrays (PatArrays)**. This allows native, structural lexical matching with a fraction of the memory footprint.
•**Hardware-Fused Vector Traversal:** By leveraging a strict 24-bit metadata ceiling embedded directly into native 64-bit coordinate labels, CoreQuarry unifies hard multi-tenant boolean constraints and soft category re-ranking weights straight inside high-performance HNSW graph loops—bypassing the slow post-filtering and cross-index stitching that plagues other engines.

•**Saturated Hardware Execution Pipelines:** While competitors rely on slow, data-dependent branch logic inside their sorting routines, CoreQuarry's indexing pipeline transforms variable string processing into deterministic, fixed-width blocks (64-character compiled horizons). This allows our core engines to utilize **branchless SIMD register vectors (AVX2, ARM Neon, and SVE2)** and cache-localized **MSD Radix sorting** to saturate hardware memory bandwidth and achieve maximum performance.

•**Bare-Metal Tensor Execution via GGML**: Unlike direct competitors that rely on heavy python-dependent runtimes, massive ONNX dependencies, or rigid, cloud-bound inference APIs, CoreQuarry fuses its vector re-ranking engine directly with the GGML tensor library. This grants the runtime native, zero-overhead access to consumer hardware and hardware accelerators—saturating Apple Silicon Unified Memory and Neural Engines via Metal, as well as commodity x86 setups via AVX-512/AMX and specialized GPUs. By utilizing GGML's state-of-the-art quantization topologies (including advanced k-quant block methods like Q4_K_M and Q8_0), CoreQuarry supports zero-copy, direct pass-through of pre-quantized model weights. This allows enterprise deployments to immediately explore the precision-to-performance frontier of lightweight, highly compressed semantic embedding spaces, executing complex multi-dimensional similarity matrices natively inside local CPU cache domains with practically zero deployment or infrastructure bloat.

Bottom line: Where direct competitors require a distributed Kubernetes cluster of high-memory JVM/Go nodes just to coordinate semantic queries, CoreQuarry delivers deep structural lexical search and blistering vector re-ranking natively inside a single unified, lightweight, bare-metal runtime.

---

2) MULTI-MODEL & RDBMS COMPETITORS (MARKETING VECTOR EXTENSIONS)

PostgreSQL (with pgvector / pgvecto.rs)

Architecture: Traditional ACID-compliant relational database utilizing third-party extensions (pgvector for IVFFlat/HNSW or Rust-based pgvecto.rs).

Strengths:

- Universal enterprise adoption; allows unified relational data and vector storage in a single database instance.
- Strong relational/boolean filtering combined with distance operators.
- Open source (PostgreSQL License).

Comparative Weakness:

- **Not built for scale:** Row-oriented engine overhead. Every vector lookup undergoes standard PostgreSQL tuple-overhead, causing massive memory footprint bloat.
- **Cache-unfriendly:** It completely lacks hardware-sympathetic optimizations for processing dense vector layers across CPU L3 cache lines or high DDR5 memory bandwidth pipelines.
- **Weak Hybrid IR:** Combining native lexical text search (tsvector/tsquery) with pgvector results in poorly optimized execution paths; the engine doesn't cleanly unify the index spaces, leading to sub-optimal execution.

ClickHouse

Architecture: Open-source Columnar OLAP (Online Analytical Processing) engine designed for real-time analytics, natively incorporating vector search functions (e.g., Distance functions and

experimental vector indexing).

Strengths:

- **Massive Columnar Throughput:** Vector processing benefits naturally from ClickHouse's core design—data is stored contiguously in columns, allowing high-performance batch scanning.
- **Built-in Parallel SIMD:** Aggressively utilizes AVX2, AVX-512, and Neon instructions out of the box for linear column array math.
- Apache 2.0 license.

Comparative Weakness:

- **Analytics, Not IR:** ClickHouse is an analytics warehouse, not a document retrieval engine. It lacks robust, structural lexical linguistic pipelines (tokenization, complex proximity operations, and text weighting matrices).
- **Mutation Heavy:** Designed for append-mostly logging workloads. High-frequency updates or fine-grained modifications to existing documents completely tank its performance.

MongoDB (Atlas Vector Search)

Architecture: Document-based NoSQL database using a proprietary vector index layer managed primarily in its cloud-hosted "Atlas" infrastructure.

Strengths:

- Massive JSON-document development ecosystem.
- Simple API integration for standard web application stacks.
- Server Side Public License (SSPL) for self-hosted community editions.

Comparative Weakness:

- **Marketing-First Hybrid Search:** The underlying storage engine (WiredTiger) is inherently optimized for nested B-Trees. Vectors are treated as an attached metadata array rather than a core coordinate label space.
- **Memory Management Stalls:** Written in C++, but suffers from massive document serialization/deserialization overhead.
- **Cloud-Lock-In:** Peak vector search features require MongoDB Atlas, removing total control over physical volume layouts and underlying storage blocks.

CouchDB

Architecture: Erlang-based Document Store focused on Master-Master replication and eventual consistency, utilizing Lucene sidecars or basic external indexes for advanced querying.

Strengths:

- Master-Master replication architecture designed for offline-first synchronization topologies.
- Apache 2.0 license.

Comparative Weakness:

- **Severe Vector Incompatibility:** Written in **Erlang**, which handles data as immutable, boxed garbage-collected terms. Erlang's runtime model is structurally incapable of performing low-level, high-performance SIMD register vector math over raw float pointers.

•**High Latency Indirect Mapping:** Vector operations require piping data out of the main database to external sidecar search engines, completely destroying modern cache-sympathetic operations.

The CoreQuarry Competitive Edge over Multi-Model Stores: While multi-model engines (PostgreSQL, ClickHouse, MongoDB, CouchDB) market vector search capabilities, they treat vectors as secondary attribute properties attached to complex, high-overhead database row/document abstractions.

CoreQuarry bypasses this structural layout tax. By utilizing bit-packed 64-bit coordinate spaces integrated directly into hardware-sympathetic data layouts (such as our local Patricia Array variant architectures and branchless SIMD/Radix terminal pipeline loops), CoreQuarry unifies hard lexical filtering and high-speed vector re-ranking natively within the CPU cache line layer. This delivers orders of magnitude faster execution speeds with a tiny, predictable memory and infrastructure footprint.

3) CLASSICAL IR SYSTEMS

Apache *Lucene*. The Foundation for ElasticSearch, OpenSearch, Solr etc.

Architecture: Traditional text search library written in Java, utilizing an inverted index composed of rigid, immutable segment files. Terms are tracked via complex internal Finite State Transducers (FSTs) that map tokens to posting lists containing document IDs, positions, and payloads.

Strengths:

- Industry standard with an immense developer ecosystem and mature tokenization/linguistic analyzer pipelines.
- Robust support for BM25 tf-idf weighting variants and complex boolean query parsers.

Comparative Weakness:

- **The Inverted Index Storage Bloat:** Lucene's core design requires building and caching immense, high-overhead term dictionaries (FST arrays) in memory. This architecture demands massive RAM overhead just to coordinate term lookups before posting lists can even be read from disk.
- **The Runtime Memory Tax (JVM Garbage Collection):** Being written in Java, Lucene suffers heavily from heap fragmentation and unpredictable GC pauses. Under high-throughput ingestion or complex structural queries, object allocations trigger execution stalls that destroy predictable latency SLAs.
- **Aggressive I/O Segment Merging:** Because segment files are immutable, continuous document updates require massive background disk-shuffling passes (Lucene Segment Merges). These passes saturate I/O channels, starving active queries of read bandwidth.
- BM25-centric scoring
- **Limited structural awareness**
- Lucene writes new data to new, immutable segments, which are merged later. Merging is a resource-intensive, slow process.
- **Deep paging limits**: Standard paging is inefficient for deep searches, often limited to the first 10,000 documents to prevent cluster instability.

Tantivity

Architecture: A modern, high-performance classical search engine library written in Rust, heavily inspired by Lucene's inverted-index architecture but built for native code execution. Follows Lucene's fundamental architecture and algorithms, including BM25 scoring and similar indexing

strategies, but takes advantage of Rust's memory safety and performance characteristics.

Strengths:

- **Native Speed & Safety:** Written in Rust, completely bypassing Java's JVM garbage collection tax and maximizing thread utilization. Much faster than Lucene.
- Strong memory-mapping (mmap) alignment for faster data access.
- MIT License.

Comparative Weakness:

- **The Legacy Structural Trait:** While Tantivy fixes the runtime language penalty of Lucene, it still retains the classical inverted-index design. Term lookups are bound to complex bit-packing schemas (like SIMD-BP128) over massive term dictionaries, retaining high memory footprint ratios.
- **Weak Native Vector/Hybrid Fusion:** It is strictly a classical text engine first. It lacks deep, hardware-fused integration between its lexical components and multidimensional vector topologies, forcing developers to manage external coordination for true hybrid semantic ranking.

Xapian

Architecture: A highly mature, open-source C++ search engine library utilizing a traditional Probabilistic Information Retrieval model (BM25/Chert/Glass storage layers built on B-Trees).

Strengths:

- Lightweight C++ compilation footprint with zero runtime dependency weight.
- Highly reliable for traditional, deterministic text indexing workloads.
- Interesting academically (long history)
- **GPL v2 License (which can also be a weakness)**

Comparative Weakness:

- **Stuck in the Pre-SIMD Era:** Xapian's core index layouts are bound to traditional, deep B-Tree node lookups. This legacy structural design is completely incompatible with modern, cache-sympathetic vector processing, hardware-fused registers, or parallel GPU/tensor architectures.
- **No Vector Architecture Capability:** Completely lacks native support for Approximate Nearest Neighbor (ANN) graphs, dense embeddings, or hardware-accelerated quantization passes.

The CoreQuarry Competitive Edge over Classical Inverted Engines

While classical IR systems (Lucene, Solr, Tantivy, Xapian) excel at parsing traditional, keyword-driven queries, they are shackled to an index-design philosophy that is fundamentally incompatible with modern hardware-sympathetic engineering. Their reliance on massive inverted index structures, immutable segment files, and heavy term-dictionaries forces infrastructure footprints to balloon rapidly as collections grow.

CoreQuarry completely breaks away from these classical design traps:

- **Bypassing the Term Dictionary Footprint:** Instead of burning gigabytes of RAM on memory-resident FSTs or complex B-Trees just to locate words, CoreQuarry utilizes a proprietary, highly optimized variant of **Patricia Arrays (PatArrays)**. This stores terms linearly and contiguously alongside a parallel vector array of address offsets (GPTYPE) mapped straight from a read-only

virtual memory-mapped text buffer.

- **Branchless Hardware-Fused Sorting Networks:** While classical engines iterate through string pointers byte-by-byte via data-dependent branch loops—stalling modern CPU execution pipelines—CoreQuarry transforms string comparisons into deterministic, fixed-width blocks (64-character compiled horizons). This allows our core ingestion engine to utilize platform-specific **branchless SIMD instructions (AVX2, ARM Neon, SVE2)** and cache-localized **MSD Radix sorting** to saturate local memory bandwidth and achieve elite ingestion throughput.
- **Zero Run-Time Stalls:** By operating strictly within a lightweight, bare-metal C++ domain and embedding its vector capabilities directly within the **GGML tensor library**, CoreQuarry completely eliminates the garbage collection pauses of Lucene/Solr and the cross-index stitching delays of Tantivy. It handles structural text filtering, tensor re-ranking, and quantization pass-through natively inside local CPU cache lines.

4) NATIVE VECTOR COEFFICIENT & ANN GRAPH COMPETITORS

FAISS (Facebook AI Similarity Search)

Architecture: A highly influential, heavy-duty C++ vector similarity library from Meta designed for dense vector clustering, utilizing Product Quantization (PQ) and optimized GPU execution paths.

Strengths:

• Exceptional handling of massive, billion-scale static vector matrices on multi-GPU server rigs.
• Deep academic optimization for vector space quantization variants (IVF-PQ).

Comparative Weakness:

• **Not a Real-Time Index Store:** FAISS is a mathematical matrix math tool, not a dynamic information retrieval system. It lacks native support for real-time CRUD operations—inserting or deleting single document vectors dynamically requires expensive, full-index re-clustering passes.
• **Zero Lexical Integration:** It possesses no concept of text metadata filtering or structural document rules. Combining lexical search with FAISS forces developers to build independent split-index architectures, suffering from high latency during post-query intersection.
• **Slower than Quarry**

ScaNN (Scalable Nearest Neighbors by Google)

Architecture: Google's state-of-the-art vector search library optimized for maximum Anisotropic Vector Quantization and hardware-specific SIMD execution.

Strengths:

• Industry-leading recall rates per millisecond due to highly advanced quantization math that minimizes quantization noise in the direction of data points.
• Deeply integrated into high-end Python and TensorFlow machine learning pipelines.

Comparative Weakness:

• **Rigid C++ Monolith:** ScaNN is notorious for its highly monolithic, complex bazel-based compilation matrix. It is deeply tightly coupled to the TensorFlow/Bazel ecosystem, making it a

nightmare to embed cleanly as a lightweight native library inside independent engines.
• **Designed for Static Batch Ingestion:** Optimized for static, mass-computed data lakes. Like FAISS, it behaves poorly under high-frequency dynamic update pipelines where document visibility must alter instantly.

HNSWlib (The Baseline Implementation for CoreQuarry's HNSW)

Architecture: The standard, open-source reference implementation of Yu. A. Malkov's Hierarchical Navigable Small World graphs for Approximate Nearest Neighbor (ANN) search.

Strengths:

• Simple, lightweight header-only C++ foundation with high recall accuracy.
• Supports real-time element insertion and multi-threaded graph building.

Comparative Weakness:

• **Unquantized Memory Ingestion:** In its baseline state, standard HNSWlib forces vectors to sit raw in memory as uncompressed floating-point streams (float32). For massive datasets, this causes immense RAM consumption, quickly exhausting L3 cache lines and bottlenecking performance at the DDR5 memory lane boundary.
• **Naive Scalar Loops:** Lacks tailored, platform-specific hardware acceleration beyond compiler auto-vectorization primitives, leading to missed optimization opportunities on modern chipsets.

The CoreQuarry Competitive Edge over Specialized Vector Backends

While specialized vector engines (FAISS, ScaNN) deliver great mathematical matrix metrics on static datasets and standard HNSWlib offers a clean reference structure, they fall completely flat when asked to act as a dynamic, resource-efficient hybrid knowledge engine. They either treat memory as an infinite resource or isolate vector execution entirely from the accompanying text documents.

CoreQuarry completely redefines high-performance vector processing:

• **The Turbo-Quantization Breakthrough:** Standard HNSWlib chokes on raw float32 bloat, forcing the CPU to constantly leave its high-speed L3 cache to fetch data from memory lanes. CoreQuarry resolves this by integrating advanced block quantization topologies. By shrinking massive high-dimensional vectors down to dense, low-bit spaces, our graph nodes live natively within local CPU cache domains, delivering orders-of-magnitude reduction in RAM footprint.
• **Zero-Copy Tensor Pass-Through:** While FAISS and ScaNN trap you inside specialized, rigid data formats that require extensive translation layers, CoreQuarry integrates directly with the **GGML tensor ecosystem**. We support native pass-through of pre-quantized state-of-the-art model weights. The vectors used for graph traversal are the exact same bytes fed to your embedding pipeline—completely cutting out runtime serialization penalties.
• **Turbo-Charged Neon Acceleration:** We didn't just compile HNSWlib; we stripped out its generic, scalar loops and re-engineered its core calculation kernels from the ground up. Optimized specifically for modern hardware like Apple Silicon MacBooks and server-side AWS Graviton rigs, CoreQuarry leverages highly unrolled, explicit **ARM Neon and SVE2 intrinsics** for its distance operations. By pairing this vector compute layer with our custom, cache-localized **MSD Radix terminal sorting pipelines**, CoreQuarry achieves blazing-fast traversal metrics on everyday consumer hardware and server clusters alike.
• **Unified Lexical Fusion:** Most importantly, CoreQuarry eliminates the "two-database problem." By embedding tight, bit-packed metadata filters directly into our 64-bit HNSW graph coordinate labels and utilizing our lightweight **Patricia Array (PatArray)** text mechanics, hard structural filters and soft semantic similarity vectors are resolved simultaneously inside a single, unified, ultra-fast traversal

loop.

---

4) HIERARCHICAL & SEMI-STRUCTURED DOCUMENT ENGINE COMPETITORS

MarkLogic

Architecture: A proprietary, enterprise NoSQL document store that commercialized the "Universal Index" concept for XML and JSON. It tokenizes and indexes both text and structural hierarchy (elements, properties, namespaces) simultaneously, enabling schema-agnostic querying via XQuery and Xpath.

Strengths:

- **Elite Structural Lexical Awareness:** Robust, schema-free structural lexical awareness for highly nested document hierarchies
- **Fully ACID-compliant** transactional guarantees across document, range, and RDF triple indexes simultaneously.

Historical Context & Structural Flaws:

•**The Commercialized Philosophy**: While MarkLogic popularized this pattern in the 2000s, the underlying philosophy of fusing structural markup and lexical tokens into a single search space was pioneered decades earlier by the Waterloo OpenText engine in the late 1980s/early 90s.

Comparative Weakness:

• **Immense Commercial and System Overhead:** MarkLogic is a massive database monolith that requires significant licensing costs and substantial hardware resource footprints. It is entirely designed to be run as an independent, heavy server cluster—not a lightweight embedded process.
• **Memory-Resident Footprint Bloat:** To achieve its fast range and lexicon lookups, MarkLogic relies heavily on massive in-memory-mapped file allocations (List Caches, Range Caches). This design lacks the strict hardware-sympathetic cache localization needed to prevent CPU starvation on consumer-grade hardware or edge devices.
• **Weak Native Vector Pipeline Acceleration:** While it handles traditional metadata text matrices, geospatial lookups, and RDF triple relationships efficiently, it lacks a modern, hardware-fused tensor core compute pipeline (like ARM Neon/SVE2 or AVX-512 graph traversals) to handle high-density embedding topologies out of the box.

BaseX

XML structural search.

Weakness:

- no vector search
- limited IR capabilities

eXist-db

Similar category.

The CoreQuarry Competitive Edge over Hierarchical Document Stores.

MarkLogic proved that treating text tokens and document structure as a single, unified search problem is the gold standard for complex information retrieval. While MarkLogic built a heavy, closed-source enterprise

database server around structural text indexing, they were commercializing a paradigm established decades earlier by the pioneering *"**OpenText engine**"* and the foundational computer science of **Ricardo Baeza-Yates and Gaston H. Gonnet**.

CoreQuarry traces its direct lineage back to this golden era of high-performance IR. Witnessing the strengths of the early Waterloo architectures and explicitly taking their work on structural algorithms to the next level within Isearch, CoreQuarry strips away the multi-node server bloat and fuses this elite structural intelligence straight into modern CPU registers:

- **Lightweight, Native Patricia Array Mechanics:** Where MarkLogic burns megabytes of RAM on sprawling universal index lists and heavy tree structures, CoreQuarry utilizes a highly compact, custom implementation of **Patricia Arrays (PatArrays)**. By mapping terms linearly and contiguously alongside a parallel vector array of address offsets (GPTYPE) from a read-only virtual memory-mapped text buffer, we achieve deep structural lexical matching with a fraction of the memory footprint.
- **Branchless Sorting Networks vs Heavy Cache Layers:** MarkLogic relies on massive system memory caches to keep its lexicon lookups fast. CoreQuarry shifts the heavy lifting to the CPU registers. By enforcing a deterministic, 64-character compiled compilation horizon, our ingestion engine converts variable text processing into fixed blocks. This allows us to leverage **branchless SIMD instructions (AVX2, ARM Neon, SVE2)** and cache-localized **MSD Radix sorting** to saturate hardware memory bandwidth and achieve elite throughput.
- Modern RAG and genomic workloads demand more than just classical text hierarchies.
- **Fused Vector/Tensor Acceleration via GGML:** MarkLogic can orchestrate complex metadata and triple relationships, but it forces you into high-overhead data boundaries. CoreQuarry integrates an embedded **GGML tensor library backend** right into the core of its data traversal pipelines. By compressing high-dimensional vectors into low-bit quantized spaces and embedding bit-packed structural criteria directly inside our 64-bit HNSW graph coordinate labels, we resolve hard structural text filters and soft semantic tensor calculations simultaneously in a single, low-latency instruction loop.

The Bottom Line: CoreQuarry represents the ultimate modernization of the classical structural search philosophy. It takes the architectural ideals pioneered by the Waterloo Oxford English Project and OpenText, strips away decades of enterprise database bloat, and delivers an ultra-compact, high-performance embedded C++ library that handles structural layouts, long-tail sequence lookups, and tensor quantization passes natively inside your application's local process space. CoreQuarry delivers the elite structural lexical intelligence of systems like MarkLogic without the enterprise bloat.

5) EMBEDDED DATABASE COMPETITORS (IN-PROCESS ENGINES)

SQLite (with FTS5 / sqlite-vec)

Architecture: The gold standard for embedded, row-oriented transactional (ACID) databases. Text search is handled via the FTS5 extension (inverted index), and vector capabilities are retrofitted via third-party extensions like sqlite-vec. Many RAG systems are now built on top of SQLite + vector extensions.

Strengths:

• Ubiquitous, hyper-lightweight, and bulletproof reliability with an unmatched deployment footprint.
• Excellent for transactional metadata handling and structured local configuration storage.
• Public Domain license.

Comparative Weakness:

• **Row-Store Bottleneck:** SQLite's core architecture is built around B-Trees optimized for small row mutations. High-dimensional vectors or mass document blocks force massive page-splitting and intensive serialization overhead.
• **Naively Attached Extensions:** Both FTS5 and sqlite-vec operate as virtual table abstractions. Because they are bolted *onto* SQLite rather than fused into its core, running a hybrid query (e.g., matching a keyword while scanning an ANN graph) requires slow, multi-pass row filtering that prevents unified, cache-line pipeline execution.

DuckDB

Architecture: An embedded, columnar analytical (OLAP) database engine written in C++, designed for high-performance vector-vector processing, SQL query execution, and large-scale data analysis tabs.

Strengths:

• Blistering speed for columnar math, aggregations, and scanning structured parquet/CSV datasets.
• Deeply sympathetic to modern multi-core hardware, utilizing vectorized execution pipelines natively.
• MIT License.

Comparative Weakness:

• **Analytical, Not Information Retrieval:** DuckDB is an OLAP data-warehouse engine, not an information retrieval system. It completely lacks specialized, structural lexical linguistic pipelines (proximity scanning, positional matrix relevance, and field-weighted tracking arrays).
• **Transient Ingestion Model:** Designed for read-heavy analytical workloads on relatively static data. High-frequency, random incremental document text updates completely degrade its performance and fragment its columnar memory blocks.

The CoreQuarry Competitive Edge over Embedded Engines

While SQLite and DuckDB dominate the embedded database ecosystem, they are general-purpose relational and analytical platforms. SQLite treats information retrieval as an isolated virtual table extension, while DuckDB views data exclusively through a data-warehouse column matrix. Neither is built to handle the complex, low-latency demands of real-time hybrid knowledge discovery.

CoreQuarry delivers a purpose-built architectural alternative:

- **Unified Memory-Mapped Text Spaces:** Unlike SQLite, which pays a massive serialization and B-Tree indexing tax on text fields, CoreQuarry maps an unfragmented, read-only text buffer directly to virtual memory. Our terminal pipelines run straight against this raw data via highly compact **Patricia Array (PatArray)** structures, eliminating document-to-row transformation delays completely.
- **Hardware-Fused Core Engines:** Where SQLite coordinates separate extensions via high-overhead virtual interfaces, CoreQuarry unifies hard text restrictions and soft tensor re-ranking metrics natively. By embedding bit-packed metadata criteria straight inside our 64-bit HNSW graph coordinate labels, we execute complex hybrid searches in a single instruction pass.
- **Saturated Vector/Tensor Pipelines:** DuckDB excels at processing standard columns but lacks specialized text-sorting architectures. CoreQuarry pairs an embedded **GGML tensor library backend** with an explicitly unrolled **ARM Neon/SVE2 distance kernel**. Combined with our **MSD Radix string sort**—which shapes variable word layouts into fixed, deterministic 64-character compilation horizons—CoreQuarry saturates the CPU's local cache boundaries to deliver

unparalleled indexing and query performance on bare metal.

The Bottom Line: If your application needs to compute millions of financial rows or handle standard relational app storage, use DuckDB or SQLite. But if your mission is to embed an elite, low-footprint, bare-metal knowledge stack that effortlessly fuses deep structural text search, massive genomic/long-string sequence matches, and lightning-fast tensor vector re-ranking directly inside your application process, **CoreQuarry stands completely alone.**

6) APPLICATION ORCHESTRATION ARCHITECTURES (RAG PLUMBING)

These are indirect competitors, because they often define the retrieval stack. **They** are plumbing frameworks; they do not solve the underlying hardware efficiency, memory consumption, or retrieval bottlenecks.

LangChain

Architecture: A broad, generic orchestration framework designed to string together LLMs, prompt templates, memories, and external tools via a modular, graph-based or chain-based execution abstractions (LCEL - LangChain Expression Language).

Strengths:

• Massive integration ecosystem—can connect almost any LLM provider to any database, API, or vector store.
• Highly flexible for building autonomous multi-agent systems and nondeterministic workflows (via LangGraph).
• MIT License.
• Often integrates with: FAISS, Pinecone or Qdrant

Comparative Weakness:

• **The Abstraction Tax:** Heavily layered with nested, convoluted abstractions. A single retrieval step can traverse dozen of Python class wrappers, making debugging, profiling, and optimizing execution latency incredibly difficult.
• **Performance Indifference:** Written primarily in Python/TypeScript, it makes zero attempts to optimize local hardware pipelines. It relies completely on network I/O or external server-side databases to handle actual data operations.

LlamaIndex (formerly GPT Index)

Architecture: An orchestration framework explicitly focused on data ingestion, data structuring, and context augmentation for LLMs. It partitions unstructured text into structural Node objects and builds intermediate relational layouts (Vector, Summary, Tree, Graph).

Strengths:

• Elite, out-of-the-box data parsers and connectors (LlamaHub/LlamaParse) for complex enterprise formats like PDFs and slide decks.
• Clean, high-level abstractions designed specifically for context retrieval and response synthesis patterns.
• MIT License.

Comparative Weakness:

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

- **High Ingestion Overheads:** Like LangChain, its core data manipulation pipelines run in Python. Converting gigabytes of raw text documents into Python Node dictionary objects creates massive memory inflation and execution slowness during ingestion.
- **Shallow Internal Data Management:** While it supports complex data structures concept-side, it doesn't possess a high-performance local storage engine. For production data sizes, it offloads actual data operations to third-party vector databases, inheriting all their structural baggage.

The CoreQuarry Competitive Edge over Inefficient Orchestration Tooling:

LangChain and LlamaIndex have popularized the RAG architectural pattern, but they are enterprise orchestrators operating at the topmost layer of the application software stack. They solve the problem of *how to link an LLM API to a data source*, but they treat the underlying retrieval step as a slow black box—forcing applications to wade through layers of heavy Python abstractions, JSON serialization penalties, and multi-database infrastructure dependencies.

CoreQuarry bridges the chasm between orchestration logic and hardware performance:

- **Replacing the Python Abstraction Tax with Bare-Metal Speed:** While LlamaIndex and LangChain inflate an application's RAM usage by converting text fields into complex object trees, CoreQuarry operates entirely within a unified, dependency-free **C++ native environment**. We ingest, parse, and partition documents with zero serialization overhead, mapping data straight to virtual memory pools.
- **Fused Data & Tensor Execution:** To implement a basic hybrid RAG loop, LangChain must coordinate a keyword query against one database (like Elasticsearch), run a vector query against another database (like Pinecone), and stitch the results back together in Python code. CoreQuarry completely cuts out this cross-network coordination layer. By embedding bit-packed metadata restrictions directly into our **64-bit HNSW graph coordinate labels** and running lexical passes through our compact **Patricia Array (PatArray)** mechanics, hard text filtering and soft semantic tensor calculations are resolved simultaneously in a single, low-latency execution pass.
- **Hardware-Sympathetic Sorting & Inference**: While orchestration frameworks leave vector distance computations and list transformations to external engines, CoreQuarry links its graph traversal directly to an embedded GGML tensor library backend. Our distance loops are explicitly accelerated via custom ARM Neon/SVE2 intrinsics and paired with our MSD Radix string sort. By shaping variable term spaces into fixed, deterministic 64-character compilation horizons, CoreQuarry saturates the host CPU's cache line bounds to deliver lightning-fast retrieval speeds on everyday consumer hardware and bare-metal server clusters.

CoreQuarry does not necessarily replace LangChain or LlamaIndex for complex multi-agent system state management; rather, it makes them obsolete for data-heavy local environments. Instead of deploying a multi-node Kubernetes cluster of vector stores, lexical engines, and Python chunking servers just to pipe context to a local model, CoreQuarry provides a unified, ultra-compact, high-performance embedded data engine that handles the entire retrieval and tensor quantization cycle natively within a single host process.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Architectural & Competitive Breakdown

Competitor Tier / Paradigm	Representative Systems	Architectural Tax	Language / Runtime Issues	CoreQuarry's Advantage
Classical Inverted IR	Apache Lucene, Elasticsearch, Solr, Tantivy	Immense RAM burn for memory-resident FST term dictionaries; I/O-heavy background segment merges.	Java JVM fragmentation and unpredictable Garbage Collection stalls (Lucene/Solr).	**PatArray Architecture:** Elimination of traditional inverted dictionary bloat via cache-contiguous, virtual memory-mapped pointer tracking layouts.
Hybrid & Vector First DBs	Weaviate, Qdrant, Milvus	Vectors-first focus leaves them with weak lexical capabilities; require separate post-filtering cycles or cross-index stitching.	Go runtime memory isolation lag; massive multi-node infrastructure dependencies (Kubernetes).	**Hardware-Fused Co-Execution:** Unifies hard lexical masks and soft tensor weights natively inside single-pass graph traversal runs.
Specialized Vector Backends	FAISS, ScaNN, Baseline HNSWlib	No native transactional CRUD (FAISS/ScaNN); Baseline HNSWlib forces unquantized float32 bloat that exhausts CPU L3 cache pipelines.	Monolithic build matrices (ScaNN/Bazel); lack of native metadata layer capabilities.	**Turbo-Quantization Primitives:** Blistering, explicitly unrolled ARM Neon/SVE2 kernels driven by compact, low-bit block quantization layouts.
Embedded Databases	SQLite (with extensions), DuckDB	Row-store overhead/serialization penalties (SQLite); analytical column matrices entirely lack structural proximity and linguistic IR paths (DuckDB).	C++ native but bound to isolated virtual table bridges or transient batch-ingestion models.	**In-Process Fusion:** Zero serialization delays. Text boundaries, sequence configurations, and tensor spaces execute concurrently inside the host app process.
RAG Orchestration	LlamaIndex, LangChain	The "Abstraction Tax." Massive performance degradation by shifting core document segmentation and data mapping to Python object layers.	Python/ TypeScript runtime speed ceilings; rely completely on network I/O or external cluster storage engines.	**Bare-Metal Efficiency:** Fuses the entire ingestion, parsing, tensor quantization, and hybrid discovery lifecycle into a single native runtime.
Structural Lexical Search	MarkLogic	Heavy enterprise server monolith; massive memory-resident cache footprints (List/Range Caches).	Proprietary closed-source distribution; completely un-optimized for embedded edge execution topologies.	**Gonnet/Baeza-Yates Inspired Lineage:** Elevates early structured text principles to modern silicon. Uses branchless SIMD and MSD Radix sorting over fixed-width 64-character compilation horizons.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

Appendix IV: Used Technologies and main AI tech

The core is a novel multimodal search and retrieval engine. It is a kind of hybrid between full-text, XML, object and graph noSQL-db that natively ingests a wide range of document types and data formats. Lexical search is based not on inverted indexes but on Pat Arrays with a two-tier address-based caching system that solved its main limitations. Dense vector search is handled by HNSW. The implementation is based on the HNSWlib references but heavily enhanced and turbo-charged.

Behind our embeddings is our fork of bert.cpp and the mainlined llama.cpp. Behind these is the GGML tensor library. Unlike mainstream vendor libraries (like PyTorch, TensorFlow, or TensorRT) built for data-center GPUs, GGML runs large transformer models efficiently on commodity hardware.

1. **Superior CPU and Edge Device Execution**.
Mainstream libraries rely heavily on massive GPU VRAM to hold model weights during inference. GGML is written in plain C and C++, making it highly portable.

- *Zero Dependencies*: It requires no heavy third-party stacks (such as CUDA or ROCm) just to run on a standard machine.
- *Hardware Interoperability*: It is optimized for x86 architectures (AVX, AVX2, AVX-512) and Apple Silicon (via native ARM NEON and Metal).
- *System RAM Utilization*: Because most modern consumer computers feature far more system RAM than GPU VRAM, GGML natively bridges the gap by enabling models to run efficiently on standard CPUs or via CPU-GPU offloading.

2. **Accelerators**: GGML features a highly modular backend architecture. This layout allows it to offload computation graph execution to a variety of hardware accelerators via specific hardware drivers, completely bypassing heavy Python stacks.

3. **Advanced, Fine-Grained Quantization**: One of GGML's most compelling features is its pioneering approach to model quantization (compressing floating-point numbers into lower bit-widths, such as 4-bit or 5-bit).

- *K-Quants*: GGML utilizes highly engineered, variable bit-width quantization strategies (like Q4_K_M) that minimize the traditional loss in output accuracy.
- *Memory Compression*: It shrinks the memory footprint of massive models by up to 70-80% without heavily compromising the model's perplexity or reasoning capabilities.

4. **Streamlined Portability** (The GGUF Standard). GGML's associated file format, GGUF, was specifically designed to solve the fragmentation and loading issues of older ML formats.

5. **No Python Overhead**

6. **Specialized Text-Generation Ecosystem**

While mainstream libraries like TensorRT or PyTorch remain undisputed for model training and high-throughput server farms, GGML democratizes access to AI by allowing us to run powerful models locally on mainstream hardware.

While we build upon a heavily refactored bert.cpp we also support llama.cpp for more modern sophisticated models that don't use the BERT architecture such as EuroBERT.

Appendix V: Accelerators

Our lexical algorithms and HNSW implementation is, by design, CPU bound (the later using heavily optimized SIMD instructions) to leave the accelerators free to do the heavy lifting of running the models. Even on an Apple M1 (Pro) we've clocked 13k QPS-- on an M5 Pro we expect 30,000 to 39,000+ QPS-- so

we found ample state-of-the-art performance already on CPUs.

1. **NVIDIA CUDA**
   - *Quality*: This is the most mature, heavily optimized backend in the GGML ecosystem. It receives immediate day-one updates for new architectures.
   - *Performance*: Offers the highest token-per-second generation rates. It handles massive context windows and batched requests effortlessly.
   - *Efficiency*: While desktop NVIDIA cards draw significant wattage, the work-per-watt remains high because the GPU completes the generation sequence rapidly and returns to idle.

2. **Apple Metal**
   - *Quality*: Flawless integration natively recognized by macOS and iOS.
   - *Performance*: Extremely rapid. Because Apple Silicon uses a Unified Memory Architecture, the CPU and GPU share the same physical RAM pool. GGML leverages this via mmap, allowing a model to load instantly without a slow transfer over a PCIe bus.
   - *Efficiency*: Unmatched. Users can run quantized 70B models locally on a MacBook with minimal heat production, preserving battery life during background operations.

3. **Vulkan** (The Compatibility Champion)
   - *Quality*: Highly robust and serves as the best open-source, vendor-agnostic fallback.
   - *Performance*: While it slightly lags behind native CUDA or ROCm tweaks, it delivers impressive matrix-multiplication speeds across diverse hardware, including Steam Decks, Raspberry Pis, and mixed-GPU setups.
   - *Efficiency*: Balanced. Vulkan provides hardware acceleration on low-power devices that lack dedicated AI chips.

Appendix VI: Vector search performance

Speed matters because RAG pipelines are dominated by retrieval latency. Faster retrieval can dramatically improve RAG pipelines

Benchmarking on:

```
Model Name:                MacBook Pro
Model Identifier:          MacBookPro18,1
Model Number:              MK183D/A
Chip:                      Apple M1 Pro
Total Number of Cores:     10 (8 performance and 2 efficiency)
Memory:                    16 GB
System Firmware Version:   11881.140.96
OS Loader Version:         11881.140.96
```

as a baseline for a minimal probable use.

System	Approx QPS (768d)	Notes
Qdrant	~1k–3k	depends heavily on HNSW tuning
Milvus	~2k–5k	heavier infra

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

FAISS	~5k–10k	optimized C++ but limited system features
Elasticsearch	usually <2k	heavy overhead
Quarry HNSW	13k	C++.

Comparing to FAISS and single thread:

Index type	Dataset size	QPS (1 thread)	Notes
Flat (exact search)	1M vectors	40–120 QPS	brute-force dot products
IVFFlat	1M vectors	150–400 QPS	typical ANN config
IVFPQ	1M vectors	300–900 QPS	quantized vectors
HNSW (FAISS)	1M vectors	200–600 QPS	depends heavily on efSearch
Quarry HNSW	1M vectors	3000 QPS	Faster for quantized vectors

The C++ HNSWlib was already one of the fastest HNSW implementations on the market. We kept to CPU only (to allow better utilization of the accelerators for the embedding models) but turbo-charged it to be 4-6x faster with Fp32 on ARM hardware and expanded it support larger datasets through the reduction of memory demands through quantization. Comparing our optimized version with a straight compile (Clang without -O) the difference was 20x on an Apple M1. We also added support for pre-quantised models. These require not just much less memory but are also significantly faster than Fp32 models—since we implemented pass-through our HNSW graph networks also demand far less memory than solutions that pad to Fp32.

Appendix VII: Vectorization performance

We are developing and benchmarking using the M1 as a baseline upon which to base preformance expectations. Instead of demanding the latest and greatest (or an array of H200s) our design goals and constraints are to provide useful performance on literally "the smallest machine possible"— and not to demand some hyperscalar cluster.

```
model:  sentence-transformers_all-MiniLM-L12-v2
main:   token time =    0.03 ms / 0.01 ms per token
main:    eval time =   15.07 ms / 3.77 ms per token

model:  sentence-transformers_all-MiniLM-L12-v2-f32.gguf
main:   token time =    0.02 ms / 0.00 ms per token
main:    eval time =   21.52 ms / 5.38 ms per token

model:  bge-large-en-v1.5-q4_k_m.gguf
main:   token time =    0.03 ms / 0.01 ms per token
main:    eval time =   30.91 ms / 7.73 ms per token
```

Turning now to a 335-million parameter model and more exhaustive production-like benchmarks:

```
Model: bge-large-en-v1.5  arch=bert  n_embd=1024  load=150 ms Warmup=5 iters, measure=50 iters

== latency by input length (batch=1) ==
```

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

```
 very-short    tok= 11 bs= 1 | mean= 17.47 ms med= 17.47 p95= 18.01 min= 16.93 max= 18.38 |
630 tok/s    57.3 seq/s
 short       tok= 38 bs= 1 | mean= 23.97 ms med= 23.83 p95= 24.92 min= 22.97 max= 25.14 |
1586 tok/s    41.7 seq/s
 medium      tok=137 bs= 1 | mean= 44.67 ms med= 44.41 p95= 46.24 min= 43.39 max= 46.92
|    3067 tok/s    22.4 seq/s
 long        tok=263 bs= 1 | mean= 78.83 ms med= 78.07 p95= 81.44 min= 76.53 max= 82.25 |
3336 tok/s    12.7 seq/s
 max         tok=512 bs= 1 | mean= 132.63 ms med= 132.67 p95= 133.81 min=130.45
max=134.25 |    3860 tok/s     7.5 seq/s

== throughput by batch size (tok=128) ==
 bs=1        tok=137 bs= 1 | mean= 44.20 ms med= 43.95 p95= 45.58 min= 43.41 max= 46.02 |
3100 tok/s    22.6 seq/s
 bs=4        tok=137 bs= 4 | mean= 161.03 ms med= 160.01 p95= 166.50 min=156.61 max=167.15
|    3403 tok/s    24.8 seq/s
 bs=8        tok=137 bs= 8 | mean= 314.57 ms med= 313.94 p95= 324.07 min=291.38 max=328.69
|    3484 tok/s    25.4 seq/s
```

Because throughput peaks at around 3,484 tokens/sec (0.29 ms per token) we use these metrics to calculate some real-world document ingestion limits. If you block your raw text files into standard overlapping paragraph windows of roughly 256 tokens each: A single passage will compute in ~78 ms.

Our test M1-Pro chews through roughly 45 passages per second per instance (3484 tok/s÷78 ms). That means one can expect to process 2,700 fully dense semantic records per minute on a baseline Apple Silicon chip. Our tests on M3Pro and M4Pro showed even significantly higher throughputs (80k tokens/s or as much as 20x).

- M1 Pro (ARMv8): Uses an older iteration of Apple's proprietary AMX (Apple Matrix Coprocessor). It is highly optimized for accelerating single-sequence matrix-vector math (bs=1).
- M4 Family (ARMv9 + SME): The M4 introduced SME (Scalable Matrix Extension). SME is explicitly designed to handle outer-product matrix-matrix tiles natively in hardware. When you increase the batch size, the mathematical operation changes from a series of matrix-vector multiplies into a massive matrix-matrix multiplication (GEMM). The M4's SME hardware can swallow those batched matrix tiles concurrently, executing them multiple times faster per clock cycle than the M1 Pro's AMX.
- An M4 Pro has up to 20 next-generation GPU cores, and an M4 Max has up to 40. Combined with a memory bandwidth of up to 546 GB/s, the M4 Max has a massive parallel execution pool.

With a base M4 Mac mini, we can expect benchmarking metrics something like this:

- Single Query Latency (bs=1, tok=38): Will drop from your current ~24 ms down to under 8 ms, making live UI search-as-you-type operations completely instantaneous.
- Peak Batch Throughput: Will scale up from your current 3,484 tokens/second to a sustained bracket of 12,000 to 15,000 tokens/second.

The Projected M5 Performance Matrix:

- Running bge-large-en-v1.5 on a cheap M5 Mac mini will yield metrics that look more like a dedicated server data center than a compact desktop:
- Interactive Query Latency (bs=1, short query): ~3 to 5 ms (making search-as-you-type feel physically instantaneous).
- Peak Batch Throughput: Easily hitting 45,000 to 60,000+ tokens/second.

At 60,000 tokens per second, a single entry-level M5 mini will let you index up to 14,000 full paragraphs every single minute.

Quarry: Next-Generation Knowledge Retrieval Kernel for Local AI Applications.

As mentioned we support also Vukan and CUDA. As a reference machine for CUDA we've chosen the NVIDIA Jetson AGX Orin. It is an edge AI module delivering up to 275 TOPS of performance. It features configurable power consumption ranging from **15W to 75W** depending on the model, and comes in 32GB and 64GBSystem-on-Module (SOM) configurations.

```
ggml_cuda_init: found 1 CUDA devices (Total VRAM: 62796 MiB):
Device 0: Orin, compute capability 8.7, VMM: yes, VRAM: 62796 MiB
CUDA Device 0: Orin (65.85 GB VRAM)
bert: CUDA backend does not support get_rows for q4_K — embedding lookups will run on CPU, all other ops on CUDA
Model: bge-large-en-v1.5  arch=bert  n_embd=1024  load=817 ms
Warmup=5 iters, measure=50 iters

== latency by input length (batch=1) ==
 very-short    tok= 11  bs= 1  | mean=  9.32 ms  med=  8.36  p95=  12.36  min=  7.04  max= 13.72  |  1181 tok/s    107.3 seq/s
 short         tok= 38  bs= 1  | mean= 10.31 ms  med= 10.21  p95=  10.93  min=  9.85  max= 11.13  |  3686 tok/s     97.0 seq/s
 medium        tok=137  bs= 1  | mean= 19.40 ms  med= 19.35  p95=  19.90  min= 18.88  max= 20.64  |  7061 tok/s     51.5 seq/s
 long          tok=263  bs= 1  | mean= 35.19 ms  med= 35.15  p95=  35.73  min= 33.71  max= 36.83  |  7474 tok/s     28.4 seq/s
 max           tok=512  bs= 1  | mean= 59.82 ms  med= 59.79  p95=  60.74  min= 58.65  max= 61.31  |  8559 tok/s     16.7 seq/s

== throughput by batch size (tok=128) ==
 bs=1          tok=137  bs= 1  | mean= 19.36 ms  med= 19.32  p95=  19.82  min= 18.86  max= 20.60  |  7076 tok/s     51.6 seq/s
 bs=4          tok=137  bs= 4  | mean= 67.49 ms  med= 67.50  p95=  68.38  min= 65.81  max= 68.95  |  8119 tok/s     59.3 seq/s
 bs=8          tok=137  bs= 8  | mean= 135.10 ms  med= 134.98  p95= 136.11  min=133.79  max=137.25  |  8113 tok/s     59.2 seq/s
 bs=16         tok=137  bs= 16  | mean= 262.59 ms  med= 262.67  p95= 263.98  min=260.84  max=264.76  |  8348 tok/s     60.9 seq/s
 bs=32         tok=137  bs= 32  | mean= 520.58 ms  med= 520.57  p95= 521.90  min=518.51  max=522.87  |  8421 tok/s     61.5 seq/s
```

Notice the bge-large-en-v1.5 q4_K is currently not wholly supported by the ggml CUDA backend. We still despite letting the embedding lookup run on the CPU we still get ~8k tok/s.